Sunset World | Examen técnico Data Engineer

Propuesta de arquitectura, respuestas y demostración técnica
Versión: 0.3 | **Modalidad:** prueba con datos sintéticos | **Fecha:** octubre de 2026

> **Alcance y evidencia:** Este documento responde los apartados 1-8 del PDF recibido. Los layouts concretos de `leads_ventas.csv` y `fx_historico.csv` no se recibieron con el PDF; se definieron esquemas de prueba y se documentan como supuestos. No se usaron conexiones reales a Oracle, Mantis ni servicios corporativos. En esta versión se verificaron **31 pruebas Python**, calidad local y reconciliación de datos con SQLite. Los componentes Docker/PostgreSQL/Parquet/Kafka/Spark/Airflow/pgvector quedan sujetos a pruebas de integración en ambiente compatible. No se declara un benchmark a escala.

1. Ingesta Oracle 11g a Data Lake — 20 puntos

1.1 Estrategia incremental

El extractor usa `UPDATED_AT` indexado cuando existe, un límite temporal superior exclusivo y un límite inferior inclusivo. Se busca conservar consistencia y no saltarse registros con marcas temporales repetidas; un margen de recaptura recoge retrasos de origen. El resultado se escribe como **run inmutable** en Parquet y downstream aplica UPSERT por llave natural. Se registra la ventana de extracción, `run_id`, conteos, checksum/manifest y errores. La continuación no se considera publicada hasta que Parquet ha sido conciliado y el manifest queda visible. El watermark de consumo del DW se avanza **solo tras commit downstream**, para no perder intervalos.

Ruta de ejecución: `oracle/snapshot_to_lake.py` (adaptador Oracle) o `sunset/parquet.py` (CSV sin Oracle) → `/data/parquet/{fuente}/year=AAAA/month=MM/part-NNNNNN.parquet`; en modo incremental, `data/parquet/incremental/{run_id}/{fuente}/...`. El calendario es por la fecha específica de cada fuente; `/datalake/leads/year=2024/month=01/...` es equivalente al patrón que ejemplifica el PDF.

Oracle 11g: `python-oracledb` modo **Thick** con Instant Client compatible; no usar Thin porque Oracle 11g no está soportado en ese modo. Las conexiones exigen credenciales autorizadas. Lectura SELECT con predicados acotados por índice, `arraysize`, `fetchmany` y sin `SELECT FOR UPDATE`; nunca escrituras al origen ni bloqueos de fila pedidos deliberadamente. Preferir réplica de lectura y ventanas fuera de pico cuando estén disponibles.

1.2 Cambios de esquema

En zona BRONZE los campos fuente se almacenan como texto con manifest de columnas por run: agregar columnas permite absorber nuevas propiedades sin romper extracciones. Para SILVER se aplica esquema tipado/versionado; nuevas columnas opcionales se promueven con migraciones explícitas, columnas renombradas o eliminadas disparan una alerta y revisión de contrato. **No se eliminan silenciosamente columnas obligatorias.** Un catálogo de contratos describe tipo, semántica, sensibilidad, propietario y política evolutiva.

1.3 Reintentos y duplicados

Los reintentos vuelven a escribir una ventana con `run_id` derivado del intervalo; publicaciones Parquet de snapshot utilizan reemplazo de carpeta local con respaldo. Una carga a PostgreSQL utiliza `COPY` a staging temporal y `INSERT ... ON CONFLICT` en transacción por lote. Los datos sin clave natural fiable se mandan a cuarentena para conciliación; el sistema nunca presenta *exactly-once* físico por el mero hecho de tener reintentos. Un archivo fuera de orden o una actualización tardía puede requerir reproceso del rango.

1.4 Si no existe `UPDATED_AT`

Las opciones —en orden preferente, sujeto a DBA— son: (1) captura de cambios a través de redo logs/GoldenGate o un mecanismo CDC autorizado; (2) columna fuente estable y SCN/flashback o `ORA_ROWSCN` con evaluación de granularidad de bloque, permisos y limitaciones; (3) comparación por hash/PK de snapshots completos con watermark de extracción como fecha operativa; (4) barridos de historial por fecha creación más reconciliación periódica para updates/deletes. **`fecha_creacion` por sí sola no identifica actualizaciones.** Los borrados requieren CDC/tombstones o reconciliación total.

1.5 Escalar a 100 tablas

Crear registro de metadatos (`source_name`, tabla permitida, llaves naturales, marca temporal, columnas, tamaño de lote, clasificación PII, destino, prioridad, SLA, política de escritura). Instanciar jobs desde configuración; repartir por pools Airflow y concurrencia por servidor Oracle; limitar conexiones/fetch-size; ofrecer backfill acotado, DLQ/cuarentena y dashboard por fuente. No escribir 100 scripts manuales ni emitir 100 consultas simultáneas a producción.

2. Data Warehouse y modelado dimensional — 20 puntos

2.1 Granos y claves

Objeto	Grano	Clave técnica y natural
`dw.fact_ventas`	Una venta confirmada	`venta_id` natural; FKs sustitutas a dimensiones
`dw.fact_fx`	Una cotización por día y par de divisas	`(fecha, moneda_origen, moneda_destino)`
`dw.fact_interacciones`	Un comentario o interacción textual	`interaccion_id` natural + dimensión cliente/tiempo
`dw.fact_tours` (complementaria)	Un tour realizado	`tour_id`; habilita denominadores sin inventarlos
`dw.dim_cliente`	Una versión de cliente	`cliente_key` surrogate, `cliente_id` natural
`dw.dim_sala`	Una versión de sala	`sala_key` surrogate, `sala_id` natural
`dw.dim_fuente`	Una fuente de adquisición	`fuente_key` surrogate, `fuente_id` natural
`dw.dim_producto`	Un producto	`producto_key` surrogate, `producto_id` natural
`dw.dim_tiempo`	Un día	`YYYYMMDD` como clave entera; fecha única

El esquema se entrega en `sql/001_init.sql`, la carga incremental dimensional en `sql/004_marts.sql` y consultas de análisis en `sql/002_business_queries.sql` y vistas `analytics`. La dimensión cliente y sala implementan SCD tipo 2 con `valid_from`, `valid_to`, `is_current` y claves técnicas. Las ventas históricas apuntan a la versión válida según su fecha; para fuentes reales habría que asegurar un campo de vigencia de negocio, no solo fecha de ingesta.

2.2 Tasa de conversión por fuente y mes

```sql
SELECT nombre_fuente, mes, leads, ventas,
       ROUND(ventas::numeric / NULLIF(leads,0), 4) AS conversion_por_lead
FROM analytics.v_conversion_fuente_mes_provisional;
```

Advertencia: aquí se asume `ventas / leads` originados en un mismo cohort temporal. Para negocio podría ser `ventas / tours calificados`, y un desfase lead→venta obligaría a cohortes de origen o ventana de atribución. No afirmar definición oficial antes de validarla.

2.3 VPG por sala y año

```sql
SELECT nombre_sala, anio, venta_mxn, tours, vpg_provisional_mxn
FROM analytics.v_vpg_sala_anio_provisional;
```

Se asume provisionalmente `venta_mxn / tours realizados`. En industria vacacional VPG puede significar *Volume per Guest* y exigir un denominador contractual concreto (parejas/tours válidos/huéspedes). No hay especificación oficial del PDF. **Ambos indicadores quedan pendientes de definición funcional y no se presentan como KPIs aprobados.**

2.4 Huérfanos, fiscal e identidad cross-system

Ventas sin cliente/fuente/sala: bloquear carga curada, registrar `ops.rejected_record` con causa y ejecutar conciliación con responsables; no eliminar en silencio ni fabricar un cliente. Se puede asignar *Unknown Member* solo con autorización para analítica provisional y exposición separada del error.

`dim_tiempo` debe contener fecha, año/mes/trimestre natural, semana ISO, indicador laboral/festivo, año/trimestre/periodo fiscal y fecha de cierre; el inicio fiscal se parametriza en configuración y debe ser aprobado por Finanzas antes de producción. En este ejemplo se modeló ejercicio calendario en la carga inicial.

Los identificadores distintos Voltux/Share CRM/TMS se resuelven en `ops.identity_crosswalk`, donde cada pareja `(sistema, ID externo)` apunta a ID canónico con procedencia, método de vinculación y confianza/revisión. No hacer joins por nombre o teléfono como si fuesen identificadores seguros; implementar resolución probabilística separada cuando esté autorizado.

3. Airflow — 15 puntos

3.1 Dependencias

```
extract_leads ----\
                   -> ingest_support_sources -> run_marts -> run_dbt -> validate_quality
extract_fx -------/
```

`airflow/dags/etl_sunset.py` implementa TaskFlow con fuentes centrales independientes en paralelo, metadatos compactos XCom (nunca millones de filas), captura de fallos, alerta Slack mediante webhook configurable y correo si hay SMTP autorizado. Frecuencia `0 2 * * *` en `America/Mexico_City`. Reintentos 3, retardo 5 minutos, SLA 2 horas (API Airflow **2.10.5**; en Airflow 3 se migraría a *Deadline Alerts*). Límite de 1 corrida activa y 3 tareas simultáneas. El DAG ejecuta los comandos dbt reales, no un marcador de éxito ficticio.

3.2 Backfill de 6 meses

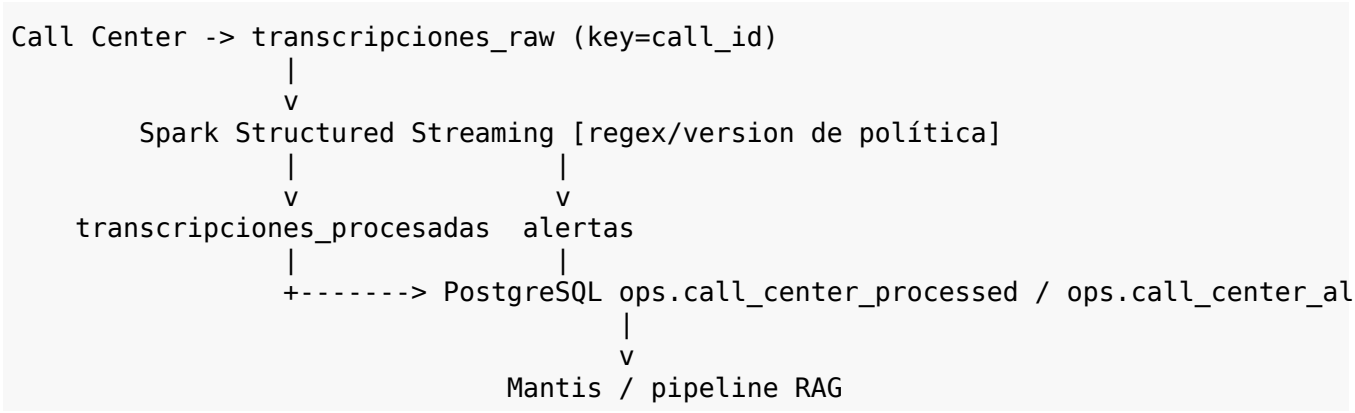
No habilitar seis meses de `catchup` masivo indiscriminado. Programar ventanas pequeñas por día y fuente, con manifiestos de rangos y prioridades, `max_active_runs=1` o pool separado, throttle de I/O y checksum por ventana. Seguir el orden temporal por fuente cuando los SCD2 dependan de orden de eventos; los hechos pueden publicarse tras dimensiones. Cada ventana es reanudable y verificable. Controlar costo, capacidad de Oracle y retención.

3.3 Evitar bloqueos y versionar

`pools`, `max_active_tasks`, colas y límites por DAG; usar sensores deferrables cuando corresponda, límites de memoria/CPU en contenedores, batch-size y SLA. Versionar DAGs en Git con code review, CI `pytest`, auditoría estática, rollback por tag y compatibilidad controlada de esquemas. **No guardar credenciales ni .env real en Git.**

4. Streaming Kafka + Spark — 15 puntos

4.1 Diseño y topics



Contrato JSON `event_id` determinista, `call_id`, `sequence`, `event_time` ISO UTC, `transcript`, `schema_version`. Con Kafka real, el `call_id` es clave de partición, permitiendo orden por partición para una llamada. **No hay orden global entre particiones**. El event-time, id único y sequence permiten gestionar mensajes tardíos/desordenados. Registrar versiones de contrato, DLQ para JSON inválidos, datos PII redaccionados, autenticación TLS/SASL en producción.

4.2 Implementación y exactly-once

`streaming/producer.py` genera eventos sintéticos y habilita idempotencia de productor. `streaming/structured_streaming.py` lee Kafka, aplica reglas configurables (`streaming/rules.py`), conserva checkpoint y escribe resultados por partición en PostgreSQL usando `INSERT ON CONFLICT(event_id)`. `foreachBatch` y sink JDBC permiten **al menos una entrega**. La clave estable vuelve *idempotente* el efecto final en PostgreSQL, pero no podemos prometer exactamente una vez para el conjunto Kafka→PostgreSQL→Kafka sin coordinación transaccional adicional; ambos topics procesados deben desduplicar por `event_id`.

4.3 Picos de tráfico, orden y Mantis

Dimensionar particiones y consumers según carga, autoscaling y backpressure de Spark, límites de retención y checkpoints en volumen persistente, alarmas por lag. Ordenar por `(call_id, sequence)` al reconstruir conversación completa; no suponer orden cross-topic. Para RAG casi real, publicar transcripciones permitidas a un índice de micro-lotes después de redacción y autorización. El índice usa metadatos de tiempo, fuente, texto y modelo para refrescar embedding; se evita exponer transcripciones sin consentimiento.

5. Calidad y monitoreo — 10 puntos

`sunset/quality.py` evalúa completitud, unicidad de claves, validez de fecha, moneda, valores monetarios, positividad de FX y frescura como regla distinta, más la auditoría de integridad referencial de todas las fuentes en `sunset/audit.py`. `dbt/models/staging/schema.yml` y `dbt/tests/` contienen tests de SQL para `not_null`, `unique`, FX positivo, fechas y ventas referenciadas. Los casos

deliberadamente inválidos en `tests/test_quality_and_offline.py` prueban que el gate detiene la publicación.

Políticas: bloquear datos que violen PK/FK, moneda o importes inválidos y adjuntar muestra de errores en cuarentena; alertar por frescura sobre fuentes históricas sintéticas (no bloquear por diseño de fixtures), bloquear en operaciones reales cuando venza SLA acordado. Métricas por lote: `extracted`, `loaded`, `rejected`, duplicados, latencia, edad máxima, errores y throughput. Para monitoreo de deriva: comparar distribuciones y características numéricas contra baseline mensual con PSI/KL según tipo de variable, muestreo y segmentación por fuente/sala, y evaluación estadística humana antes de confundir cambio de temporada con error.

Linaje: cada entidad lleva `source_system`, `table`, `file/run_id`, `source_updated_at`, `ingestion_timestamp`, versión de job y relaciones de transformaciones dbt. En producción ampliar a OpenLineage/DataHub y revisar propiedad de datos y permisos. Tablero Metabase o Grafana sobre `ops.batch_audit`, estado de DAG y calidad por fuente; nunca exponer PII en dashboards.

6. Mantis: embeddings, vector DB y RAG — 10 puntos

6.1 Arquitectura

```
Oracle/Call Center -> Parquet BRONZE -> limpieza/PII -> chunks con overlap
                                                |
                                       modelo de embeddings
                                                |
                                         PostgreSQL pgvector
                                                |
                              interfaz retrieval -> respuesta citada de Mantis
```

Elección: pgvector simplifica operaciones al coexistir con PostgreSQL, permite joins SQL, controles de acceso y transacciones. Alternativas: Pinecone (servicio administrado), Weaviate/Qdrant (motores vectoriales especializados); para una demo acotada pgvector reduce complejidad de operación. Reconsiderar a escala según volumen vectorial, latencia, privacidad y TCO.

6.2 Chunking, actualizaciones y modelo

`rag/build_embeddings.py` lee comentarios desde Parquet, elimina patrones básicos de contacto (solo demo, **no** redacción suficiente para producción), fragmenta textos con tamaño y overlap configurables, genera embeddings con sentence-transformers y los inserta en `ai.document_chunk`, dimensionalidad 384. Identifica chunks por fuente/registro/posición/modelo; calcula SHA-256 del contenido y **salta embeddings sin cambios**; reescribe los modificados, y elimina chunks obsoletos cuando se acorta el texto. `rag/search.py` devuelve fragmentos similares con `source_id` para trazabilidad. El modelo indicado es un supuesto técnico, no el modelo oficial de Mantis.

Para cambiar modelo se indexa un **nuevo `model_id`** en paralelo, se validan relevancia/latencia/cobertura en un conjunto etiquetado y luego se cambia la lectura. No mezclar vectores de diferente dimensión o modelo sin filtros. Para producción: clasificación PII, consentimiento, minimización, cifrado TLS/en reposo, aislamiento multi-tenant, RBAC por documentos, auditoría, políticas de retención/olvido, protección contra prompt injection, filtros de seguridad y evaluación de recuperación/hallucinations. Una interfaz Mantis real **no fue proporcionada**: solo se entrega el contrato conceptual de consumo del retrieval.

7. Comunicación a negocio — 5 puntos opcionales

Se incluyen `deliverables/Sunset_World_Presentacion_Ejecutiva.pptx`, versión PDF y `docs/VIDEO_SCRIPT.md`. Las cinco láminas muestran el flujo, problema, volúmenes de referencia **no ejecutados**, resultados medidos solo en muestra, mejoras de control y decisiones pendientes. El video final de cinco minutos debe grabarse después de que el repositorio se ejecute

en un ambiente compatible para mostrar Airflow, Kafka, Parquet y PostgreSQL reales, evitando fabricar capturas o resultados.

Impacto esperado sin inventar ROI: consolidación de información para FX y reporting de conversión, trazabilidad del Customer Journey, menor reconciliación manual contable y base gobernada para Mantis. Los costos requieren dimensionamiento basado en benchmark real. Supuestos críticos: autorización y SLAs de fuentes Oracle 11g, taxonomía de clientes y salas, definición VPG, condiciones de privacidad, API de Mantis y recursos de cómputo.

8. Ingeniería, seguridad y creatividad — 5 puntos

La opción A usa Docker Compose por perfiles (`demo`, `warehouse`, `lake`, `airflow`, `streaming`, `rag`), requisitos versionados, configuración `config/pipeline.json`, `.env.example`, CI GitHub Actions y `pytest`. Los conectores externos se inicializan solo cuando se invocan; la demo estándar de Python funciona sin servidor corporativo. Se entregan catálogos sintéticos con IDs compatibles, tests negativos, reconciliación de joins y reportería JSON. **Innovación propuesta, no implementada:** sustituir commits de carpetas por una tabla Iceberg/Delta para cargas concurrentes, time travel, eliminación de pequeños archivos y CDC en un despliegue real.

Registro honesto de validación

Entregable	Validación en este ambiente
Python, generador, reglas, contratos, 31 tests	Ejecutado y aprobado
Conciliación de 10 CSV, cuatro joins, FX	Ejecutado en SQLite (demo local)
Calidad de fuente	Ejecutada y aprobada en muestra histórica
Escritura/lectura Parquet real	Pendiente: `pyarrow` no instalado en este entorno
PostgreSQL/pgvector	Pendiente: servicio no disponible
Oracle 11g	Sin credenciales, esquema real ni client autorizado
Airflow/dbt Docker	Pendiente: motor Docker no disponible
Kafka/Spark Streaming	Pendiente: broker/Spark no disponibles
Mantis integración empresarial	Pendiente: contrato/API no proporcionado
Perfil de carga ~4.5M/6.9M/10M	Objetivo configurado; no se ejecutó

Próximo paso de aceptación técnica

En máquina Docker con dependencias instaladas: iniciar PostgreSQL, correr `sql/001_init.sql` y scripts de streaming, convertir todas las fuentes a Parquet, cargar `raw`, ejecutar `warehouse.build_marts` y dbt, comprobar dos ejecuciones idénticas sin duplicaciones, validar Airflow/Kafka/Spark/RAG y obtener tiempos/throughput/consumo reales; solo entonces grabar la prueba de cinco minutos y actualizar diapositivas con resultados definitivos.